

**LAPORAN
WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)
SEMESTER III**



Acara 8

Nama:

Nila Agustin Prima Aurelia

NIM:

E41250666

Golongan:

A

**PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026**

Bab 1. Pendahuluan

1.1 Latar Belakang

Dalam pengembangan aplikasi berbasis web, pengolahan data dari pengguna harus dilakukan dengan aman. Salah satu ancaman yang dapat terjadi pada aplikasi yang menggunakan database adalah SQL Injection. SQL Injection dapat terjadi apabila input pengguna langsung digabungkan ke dalam perintah SQL tanpa pengamanan.

Sebagai contoh, pengguna dapat memasukkan input:

```
' OR '1'='1
```

Apabila input tersebut langsung digabungkan dengan query SQL, maka input dapat dianggap sebagai bagian dari perintah SQL dan berpotensi menyebabkan data yang seharusnya tidak dapat diakses menjadi terbuka.

Untuk mengatasi masalah tersebut digunakan PDO (PHP Data Objects) dengan Prepared Statement. Prepared Statement memisahkan antara perintah SQL dengan data yang diberikan oleh pengguna sehingga input diperlakukan sebagai data, bukan sebagai perintah SQL.

Pada praktikum ini dibuat Sistem Informasi Akademik menggunakan PHP dengan pola MVC. Sistem memiliki tiga resource utama yaitu Mahasiswa, Program Studi (Prodi), dan Mata Kuliah. Ketiga resource tersebut digunakan untuk menerapkan operasi CRUD (Create, Read, Update, Delete) menggunakan PDO Prepared Statement.

1.2 Tujuan Praktikum

1. Memahami konsep penggunaan **PDO (PHP Data Objects)** untuk melakukan koneksi antara aplikasi PHP dengan database MySQL.
2. Memahami dan menerapkan **Prepared Statement** dalam menjalankan query SQL.
3. Menerapkan operasi **CRUD (Create, Read, Update, Delete)** pada sistem informasi akademik.
4. Menerapkan konsep **MVC (Model-View-Controller)** dalam pengembangan aplikasi PHP.
5. Menerapkan relasi antar tabel menggunakan **JOIN**, khususnya antara tabel matakuliah dan prodi.
6. Membuat fitur **pencarian data** menggunakan parameter pada Prepared Statement.
7. Menerapkan **pagination** untuk membatasi jumlah data yang ditampilkan pada setiap halaman.
8. Menerapkan validasi data mahasiswa, termasuk validasi **NIM berdasarkan tahun angkatan**.

9. Memahami penggunaan **AUTO_INCREMENT** sebagai identitas unik pada data database.
10. Menghasilkan sistem informasi akademik yang dapat melakukan pengelolaan data mahasiswa, program studi, dan mata kuliah.

Bab 2. Dasar Teori

2.1 PDO (PHP Data Objects)

PDO (PHP Data Objects) merupakan interface PHP yang digunakan untuk berkomunikasi dengan database. PDO mendukung berbagai jenis database dan menyediakan fitur Prepared Statement.

2.2 Prepared Statement

Prepared Statement merupakan mekanisme untuk memisahkan query SQL dengan nilai input pengguna. Nilai input diberikan melalui parameter sehingga dapat mengurangi risiko SQL Injection.

2.3 CRUD

CRUD merupakan operasi dasar pengolahan data yang terdiri dari:

- Create, yaitu menambahkan data.
- Read, yaitu menampilkan data.
- Update, yaitu mengubah data.
- Delete, yaitu menghapus data.

2.4 MVC

MVC (Model-View-Controller) merupakan pola arsitektur yang memisahkan aplikasi menjadi Model untuk pengolahan data, View untuk tampilan, dan Controller untuk mengatur alur aplikasi.

2.5 JOIN

JOIN digunakan untuk menggabungkan data dari beberapa tabel berdasarkan hubungan tertentu. Pada praktikum ini JOIN digunakan untuk menampilkan nama Program Studi pada daftar mahasiswa.

2.6 LIKE

LIKE digunakan untuk melakukan pencarian berdasarkan pola tertentu. Pada fitur pencarian mahasiswa, LIKE digunakan untuk mencari data berdasarkan NIM atau nama dengan Prepared Statement.

Bab 3. Hasil Praktikum

3.1 Implementasi Database Singleton dengan PDO

Implementasi koneksi database dibuat pada: app/Core/Database.php. Class Database menggunakan pola Singleton sehingga koneksi PDO hanya dibuat satu kali dan dapat digunakan kembali oleh Model.

Implementasi menggunakan: `private static ?PDO $instance = null;`

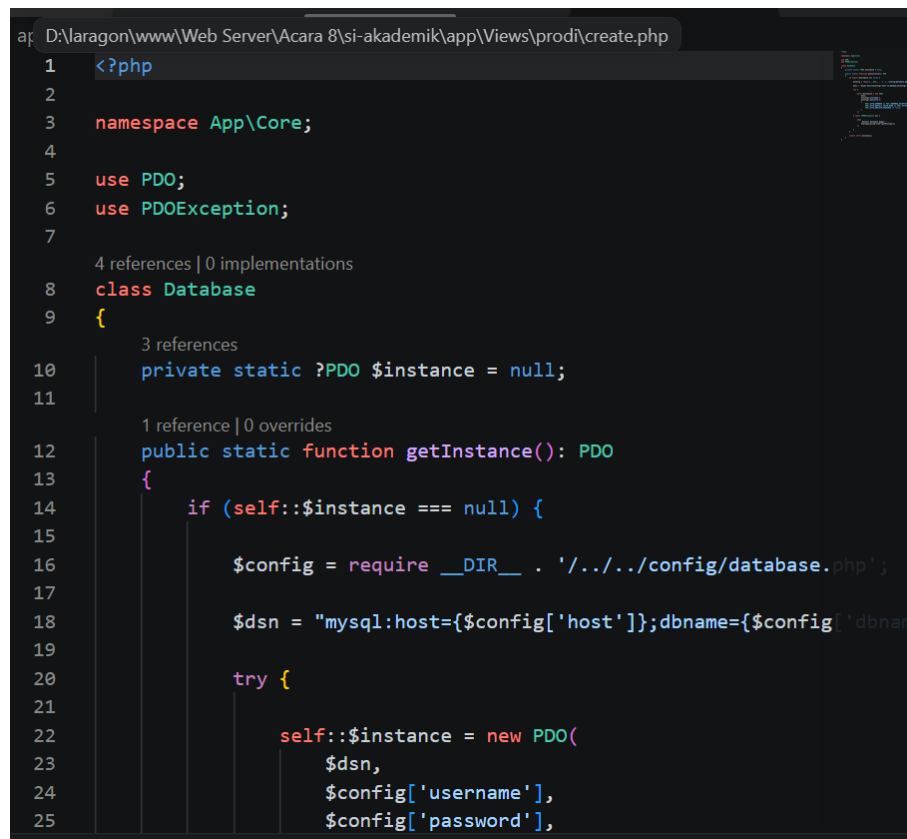
dan: `public static function getInstance(): PDO`

Koneksi dibuat menggunakan: `new PDO(...)`

serta menggunakan: `PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION`

dan: `PDO::ATTR_EMULATE_PREPARES => false`

Penggunaan `PDO::ATTR_EMULATE_PREPARES => false` mendukung penggunaan prepared statement secara lebih aman.



```
1 <?php
2
3 namespace App\Core;
4
5 use PDO;
6 use PDOException;
7
8 class Database
9 {
10     private static ?PDO $instance = null;
11
12     public static function getInstance(): PDO
13     {
14         if (self::$instance === null) {
15             $config = require __DIR__ . '/../../config/database.php';
16             $dsn = "mysql:host={$config['host']};dbname={$config['dbname']}";
17             try {
18                 self::$instance = new PDO(
19                     $dsn,
20                     $config['username'],
21                     $config['password'],
22                     [
23                         PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
24                         PDO::ATTR_EMULATE_PREPARES => false
25                     ]
26                 );
27             } catch (PDOException $e) {
28                 throw new PDOException($e->getMessage(), 150000);
29             }
30         }
31         return self::$instance;
32     }
33 }
```

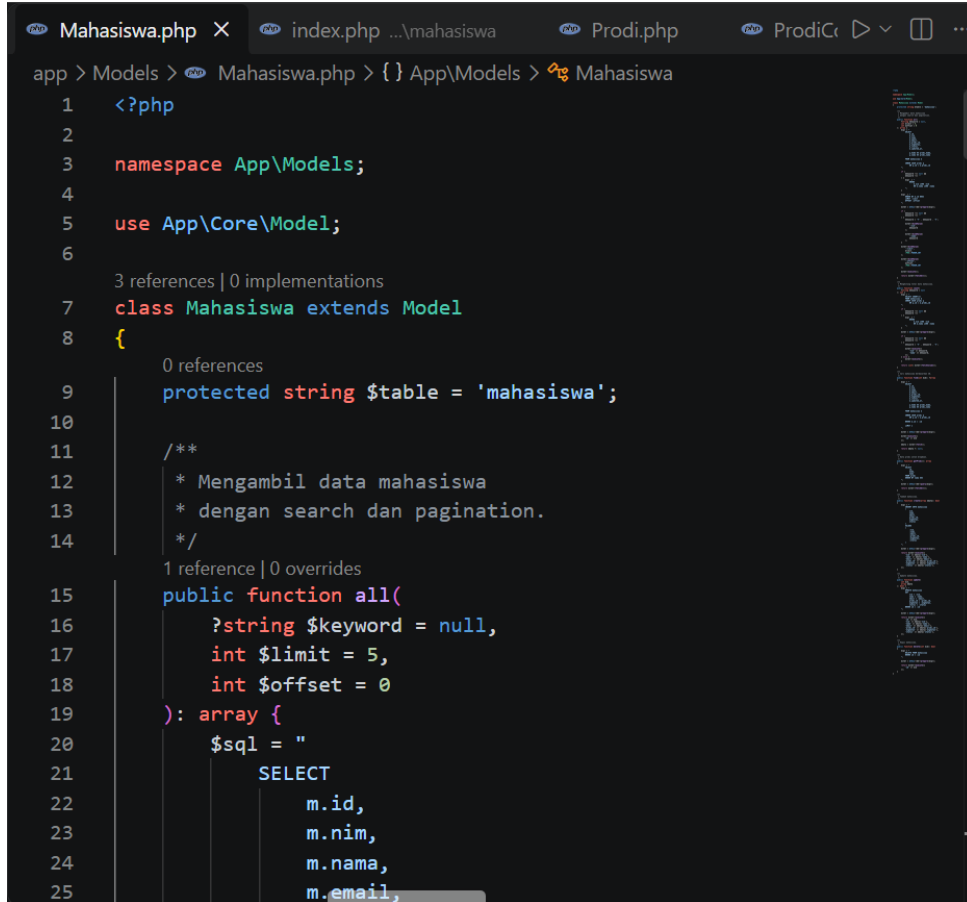
3.2 Implementasi Model

Model digunakan sebagai bagian yang berhubungan langsung dengan database. Method yang digunakan untuk pengolahan data meliputi:

- `all()` untuk mengambil seluruh data.
- `find()` untuk mengambil data berdasarkan ID.
- `create()` untuk menambahkan data.

- update() untuk mengubah data.
- delete() untuk menghapus data.

Setiap operasi database menggunakan prepare() dan parameter sehingga input tidak langsung digabungkan ke dalam query SQL.



```
app > Models > Mahasiswa.php > { } App\Models > Mahasiswa
1  <?php
2
3  namespace App\Models;
4
5  use App\Core\Model;
6
7  3 references | 0 implementations
class Mahasiswa extends Model
8  {
9      0 references
    protected string $table = 'mahasiswa';
10
11     /**
12      * Mengambil data mahasiswa
13      * dengan search dan pagination.
14      */
15     1 reference | 0 overrides
    public function all(
16         ?string $keyword = null,
17         int $limit = 5,
18         int $offset = 0
19     ): array {
20         $sql = "
21             SELECT
22                 m.id,
23                 m.nim,
24                 m.nama,
25                 m.email,
```

```
odi create.php ...prodi edit.php ...prodi MataKuliah.php X
```

app > Models > MataKuliah.php > {} App\Models > MataKuliah > all()

```
1 <?php
2
3 namespace App\Models;
4
5 use App\Core\Model;
6
7 3 references | 0 implementations
8 class MataKuliah extends Model
9 {
10     0 references
11     protected string $table = 'matakuliah';
12
13     /**
14      * Ambil data mata kuliah
15      * + JOIN prodi.
16      */
17     1 reference | 0 overrides
18     public function all(
19         int $limit = 5,
20         int $offset = 0
21     ): array {
22         $sql = "
23             SELECT
24                 m.id,
25                 m.kode,
```

```

Prodi.php X ProdiController.php index.php ...prodi create.p
app > Models > Prodi.php > { } App\Models > Prodi > all()
1  <?php
2
3  namespace App\Models;
4
5  use App\Core\Model;
6
7  3 references | 0 implementations
8  class Prodi extends Model
9  {
10     0 references
11     protected string $table = 'prodi';
12
13     /**
14      * Ambil semua prodi dengan pagination.
15      */
16     1 reference | 0 overrides
17     public function all(
18         int $limit = 5,
19         int $offset = 0
20     ): array {
21         $sql = "
22             SELECT
23                 id,
24                 kode,
25                 nama,
26                 created_at,
27                 updated_at
28             FROM prodi

```

3.3 CRUD Mahasiswa

CRUD Mahasiswa terdiri dari proses:

1. Menampilkan daftar mahasiswa.

SI Akademik

Dashboard Mahasiswa Prodi Mata Kuliah Logout

Data Mahasiswa + Tambah Mahasiswa

Cari

No	NIM	Nama	Email	Program Studi	Angkatan	Status	Aksi
1	250004	Nila Agustin Prima Aurelia	nilaaurelia97@gmail.com	TI - Teknik Informatika	2025	Aktif	Detail Edit Hapus
2	242003	Citra Lestari	citra@email.com	SI - Sistem Informasi	2024	Aktif	Detail Edit Hapus
3	251002	Ani Wijaya	ani@email.com	TI - Teknik Informatika	2025	Aktif	Detail Edit Hapus
4	251001	Budi Santoso	budi@email.com	TI - Teknik Informatika	2025	Aktif	Detail Edit Hapus

Menampilkan 4 dari 4 data mahasiswa.

← Kembali ke Dashboard

2. Menambahkan mahasiswa.

← ↻ localhost/Web%20Server/Acara%208/si-akademik/public/mahasiswa/create

SI Akademik Dashboard Mahasiswa Prodi Mata Kuliah Logout

Tambah Data Mahasiswa

NIM

Nama Mahasiswa

Email

Program Studi

-- Pilih Prodi --

Angkatan

2026

3. Menampilkan detail mahasiswa.

SI Akademik Dashboard Mahasiswa Prodi Mata Kuliah Logout

Detail Mahasiswa

NIM	250004
Nama	Nila Agustin Prima Aurelia
Email	nilaaurelia97@gmail.com
Program Studi	TI - Teknik Informatika
Angkatan	2025
Status	Aktif
Diperbarui	2026-09-27 20:07:14

Kembali Edit

4. Mengubah data mahasiswa.

SI Akademik Dashboard Mahasiswa Prodi Mata Kuliah Logout

Edit Data Mahasiswa

NIM

242003

Nama Mahasiswa

Citra Lestari

Email

citra@email.com

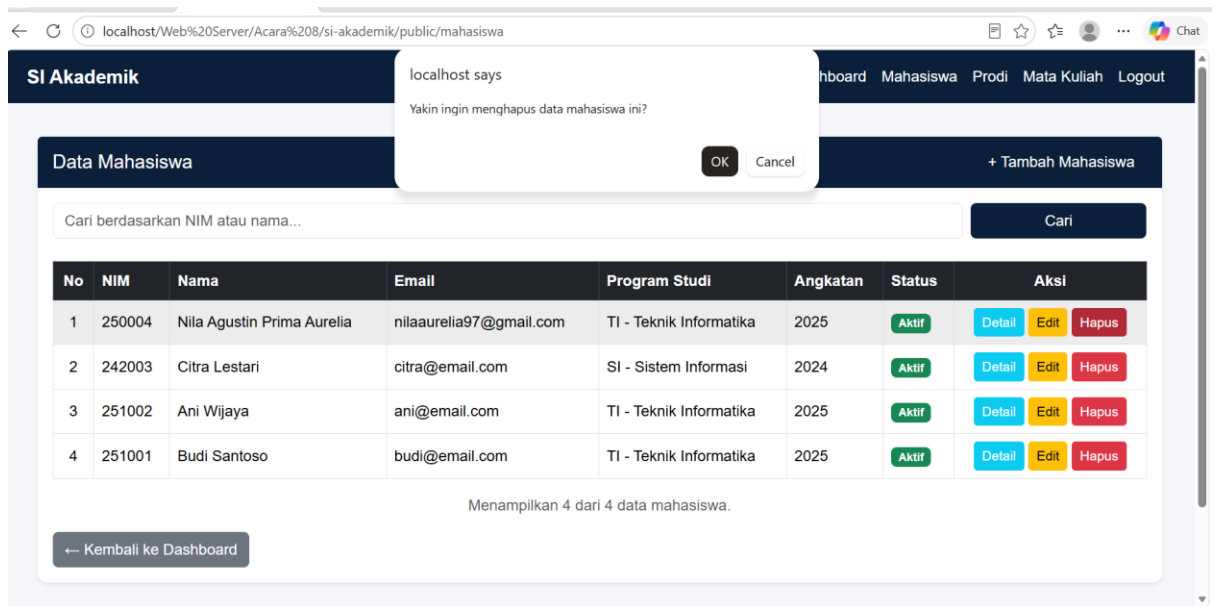
Program Studi

SI - Sistem Informasi

Angkatan

2024

5. Menghapus data mahasiswa



Controller Mahasiswa terdapat pada: `app/Controllers/MahasiswaController.php`

Model terdapat pada: `app/Models/Mahasiswa.php`

View terdapat pada: `app/Views/mahasiswa/`

View terdiri dari:

`index.php`

`create.php`

`edit.php`

`show.php`

Operasi CRUD menggunakan Prepared Statement sehingga data yang diberikan pengguna tidak langsung digabungkan dengan query SQL.

3.4 CRUD Program Studi

CRUD Program Studi digunakan untuk mengelola data Program Studi.

Implementasi terdiri dari:

`app/Controllers/ProdiController.php`

`app/Models/Prodi.php`

`app/Views/prodi/index.php`

`app/Views/prodi/create.php`

`app/Views/prodi/edit.php`

Controller menangani request pengguna, sedangkan Model menjalankan query database menggunakan Prepared Statement.

Operasi yang diterapkan adalah:

- Create Prodi.

SI Akademik

DashboardMahasiswaProdiMata KuliahLogout

Tambah Program Studi

Kode Prodi

Nama Program Studi

Kembali

Simpan

© 2026 SI Akademik - Politeknik Negeri Jember

- Read Prodi.

SI Akademik

DashboardMahasiswaProdiMata KuliahLogout

Data Program Studi

+ Tambah Prodi

No	Kode	Nama Program Studi	Aksi
1	TI	Teknik Informatika	EditHapus
2	SI	Sistem Informasi	EditHapus
3	TK	Teknik Komputer	EditHapus
4	MIK	Manajemen Informatika	EditHapus

Menampilkan 4 dari 4 data prodi.

← Dashboard

© 2026 SI Akademik - Politeknik Negeri Jember

- Update Prodi.

SI Akademik

DashboardMahasiswaProdiMata KuliahLogout

Edit Program Studi

Kode Prodi

TI

Nama Program Studi

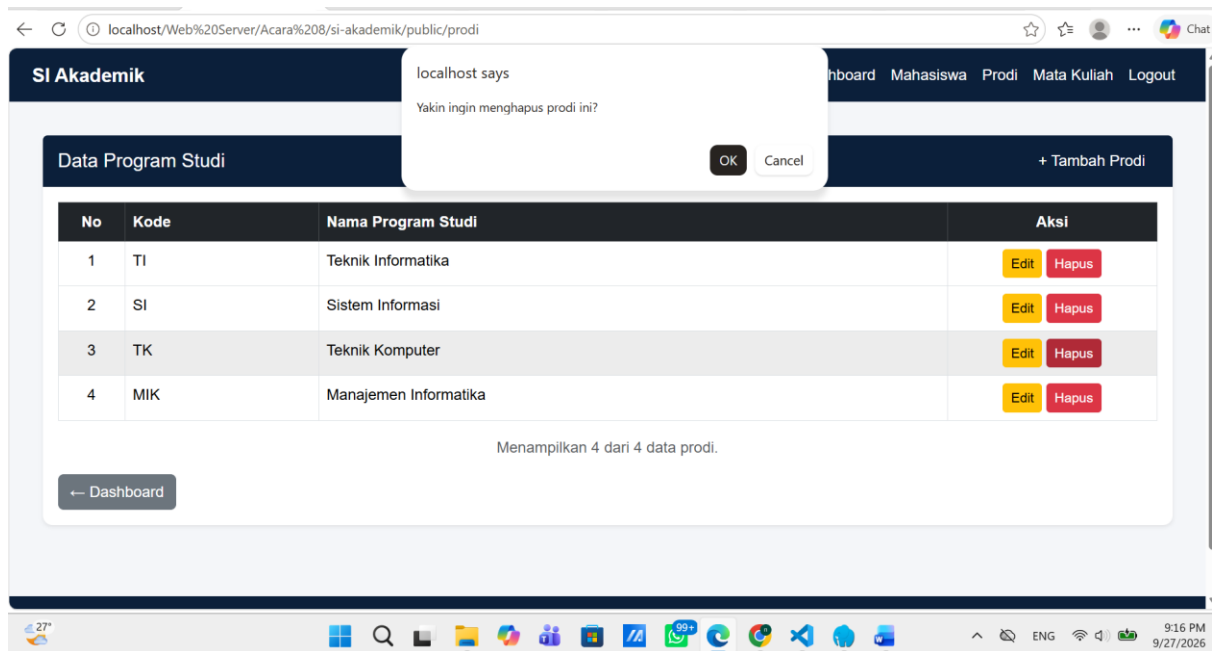
Teknik Informatika

Kembali

Simpan Perubahan

© 2026 SI Akademik - Politeknik Negeri Jember

- Delete Prodi.



3.5 CRUD Mata Kuliah

CRUD Mata Kuliah digunakan untuk mengelola data mata kuliah.

Implementasi terdiri dari:

app/Controllers/MataKuliahController.php

app/Models/MataKuliah.php

app/Views/matakuliah/index.php

app/Views/matakuliah/create.php

app/Views/matakuliah/edit.php

Data mata kuliah memiliki hubungan dengan Program Studi melalui `prodi_id`.

Operasi yang diterapkan meliputi:

- Create Mata Kuliah.

SI Akademik

[Dashboard](#)
[Mahasiswa](#)
[Prodi](#)
[Mata Kuliah](#)
[Logout](#)

Tambah Mata Kuliah

Kode Mata Kuliah

Nama Mata Kuliah

SKS

Program Studi

-- Pilih Prodi --

Kembali

Simpan

- Read Mata Kuliah.

SI Akademik

DashboardMahasiswaProdiMata KuliahLogout

Data Mata Kuliah

+ Tambah Mata Kuliah

No	Kode	Nama Mata Kuliah	SKS	Program Studi	Aksi
1	TI101	Pemrograman Dasar	3	TI - Teknik Informatika	EditHapus
2	TI102	Basis Data	3	TI - Teknik Informatika	EditHapus
3	SI101	Pengantar SI	2	SI - Sistem Informasi	EditHapus

Menampilkan 3 dari 3 data mata kuliah.

← Dashboard

© 2026 SI Akademik - Politeknik Negeri Jember

- Update Mata Kuliah.

SI Akademik

DashboardMahasiswaProdiMata KuliahLogout

Edit Mata Kuliah

Kode Mata Kuliah

TI101

Nama Mata Kuliah

Pemrograman Dasar

SKS

3

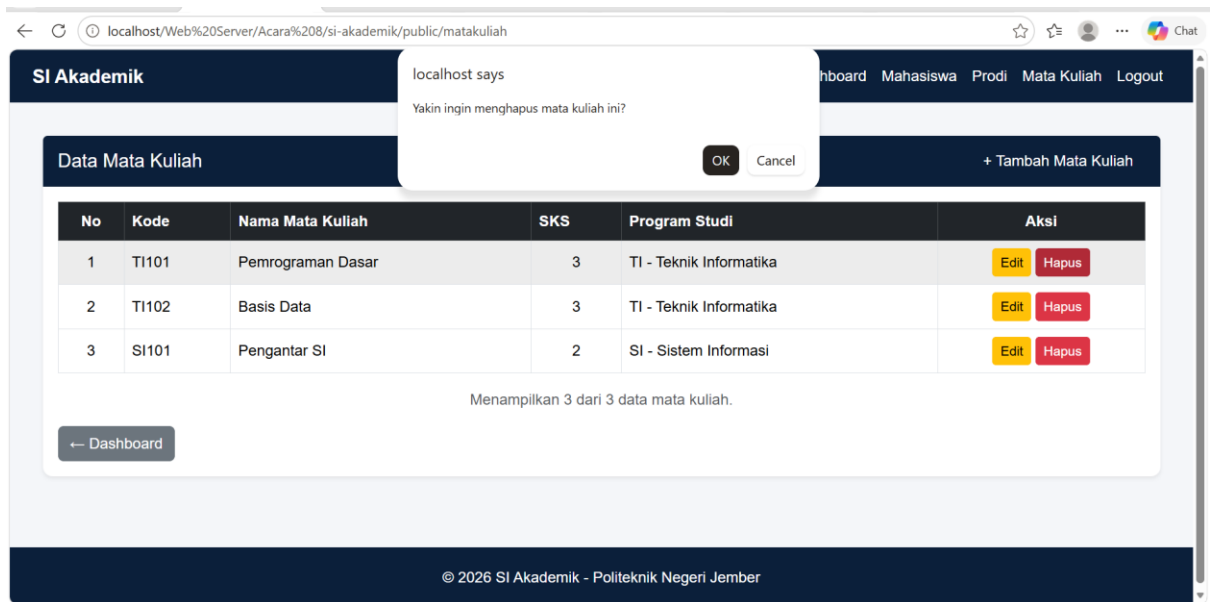
Program Studi

TI - Teknik Informatika

Kembali

Simpan Perubahan

- Delete Mata Kuliah.



3.6 Implementasi JOIN

Pada halaman daftar mahasiswa digunakan JOIN antara tabel mahasiswa dan prodi.

Query menggunakan:

INNER JOIN prodi p

ON p.id = m.prodi_id

Kemudian nama dan kode Program Studi diambil melalui:

p.kode AS prodi_kode,

p.nama AS prodi_nama

Dengan demikian daftar mahasiswa dapat menampilkan informasi seperti:

NIM | Nama | Program Studi

tanpa harus melakukan query terpisah untuk setiap mahasiswa.

Pada View, hasil JOIN ditampilkan menggunakan:

\$mhs['prodi_kode']

dan:

\$mhs['prodi_nama']

```

33
34         FROM mahasiswa m
35
36     INNER JOIN prodi p
37         ON p.id = m.prodi_id
38     ";

```

3.7 Konfirmasi Java Script Sebelum Hapus

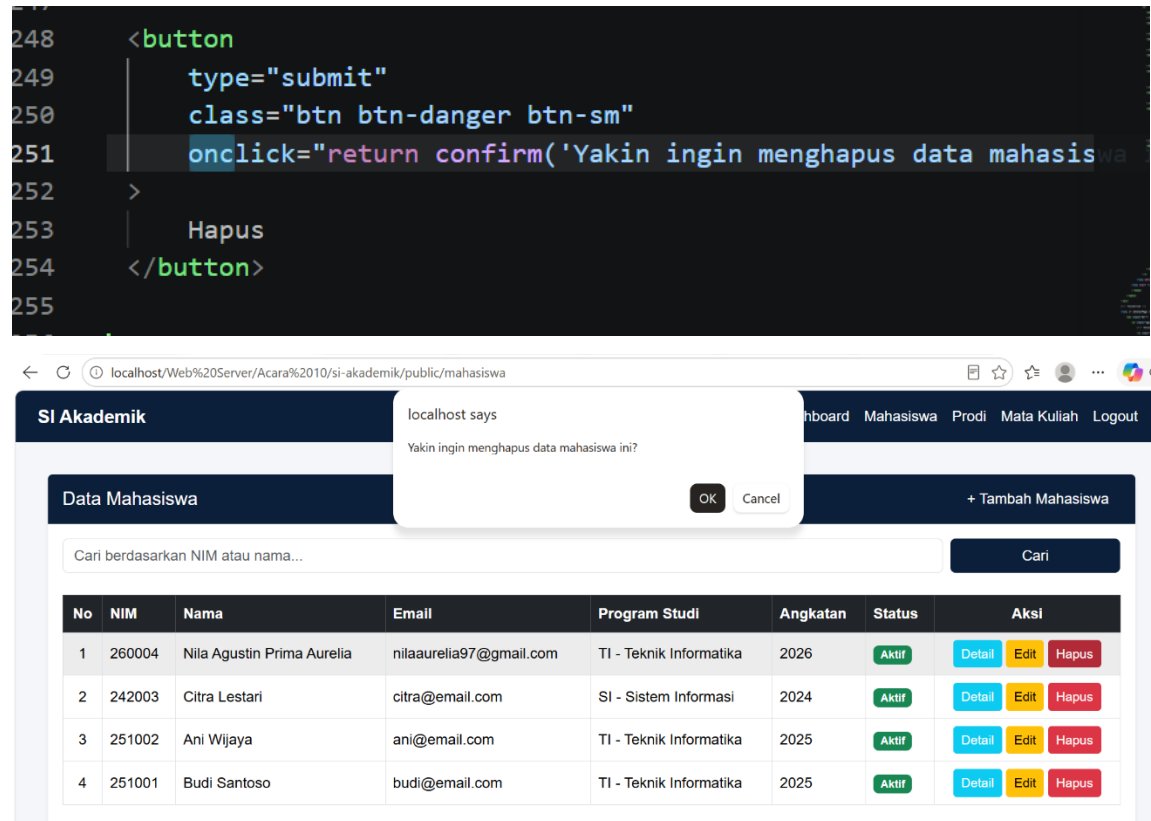
Sebelum data mahasiswa dihapus, sistem memberikan konfirmasi kepada pengguna menggunakan JavaScript.

Kode yang digunakan adalah:

```
onclick="return confirm('Yakin ingin menghapus data mahasiswa ini?')"
```

Apabila pengguna memilih OK, proses penghapusan dilanjutkan. Apabila memilih Cancel, proses penghapusan dibatalkan.

Fitur ini digunakan untuk mengurangi risiko penghapusan data secara tidak sengaja.



3.8 Fitur Pencarian

Sebagai Tugas Mandiri, ditambahkan fitur pencarian pada halaman index mahasiswa berdasarkan NIM atau nama.

Input pencarian dikirim menggunakan parameter:

```
?q=kata_pencarian
```

Controller mengambil keyword menggunakan:

```
$keyword = trim($_GET['q'] ?? "");
```

Kemudian keyword dikirim ke Model.

Pada Model digunakan kondisi:

```
WHERE
```

```
m.nim LIKE :nim
```

```
OR m.nama LIKE :nama
```

Keyword kemudian diberi karakter wildcard:

```
$keyword = '%' . $keyword . '%';
```

dan dikirim menggunakan Prepared Statement:

```
$stmt->execute([  
    'nim' => $keyword,  
    'nama' => $keyword  
]);
```

Dengan mekanisme tersebut, pencarian tetap menggunakan parameter Prepared Statement sehingga input pengguna tidak digabungkan langsung ke dalam query SQL.

```
43 {  
44     $sql .= "  
45         WHERE  
46         m.nim LIKE :nim  
47         OR m.nama LIKE :nama  
48     ";  
49 }
```

```
50 <input  
51     type="text"  
52     name="q"  
53     class="form-control"  
54     value="<?= htmlspecialchars(  
55         $keyword ?? ''  
56     ) ?>"  
57     placeholder="Cari berdasarkan NIM atau nama..."  
58 >  
59  
60 </div>
```

localhost/Web%20Server/Acara%2010/si-akademik/public/mahasiswa?q=budi

SI Akademik

Dashboard Mahasiswa Prodi Mata Kuliah Logout

Data Mahasiswa + Tambah Mahasiswa

budi Cari

No	NIM	Nama	Email	Program Studi	Angkatan	Status	Aksi
1	251001	Budi Santoso	budi@email.com	TI - Teknik Informatika	2025	Aktif	Detail Edit Hapus

Menampilkan 1 dari 1 data mahasiswa.

[← Kembali ke Dashboard](#)

© 2026 SI Akademik - Politeknik Negeri Jember

3.9 Pengujian SQL Injection

Untuk menguji keamanan Prepared Statement, dapat digunakan input:

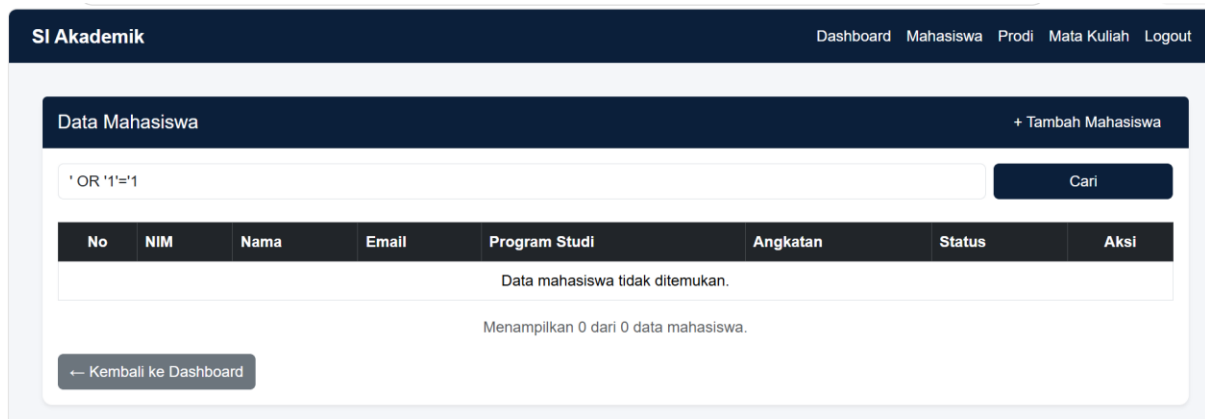
' OR '1'='1

Input tersebut dimasukkan ke form pencarian.

Karena pencarian menggunakan Prepared Statement dan parameter LIKE, input diperlakukan sebagai nilai pencarian biasa dan bukan sebagai perintah SQL.

Hasil yang diperoleh tidak menyebabkan seluruh data otomatis ditampilkan hanya karena input tersebut.

Pengujian ini menunjukkan fungsi Prepared Statement dalam memisahkan query SQL dengan data pengguna.



SI Akademik

Dashboard Mahasiswa Prodi Mata Kuliah Logout

Data Mahasiswa + Tambah Mahasiswa

' OR '1'='1 Cari

No	NIM	Nama	Email	Program Studi	Angkatan	Status	Aksi
Data mahasiswa tidak ditemukan.							
Menampilkan 0 dari 0 data mahasiswa.							

← Kembali ke Dashboard

Bab 4. Kesimpulan

Praktikum ini menerapkan PDO dengan Prepared Statement untuk membangun sistem informasi akademik berbasis PHP. Penggunaan Prepared Statement memungkinkan query dan data pengguna dipisahkan sehingga input pengguna tidak langsung dianggap sebagai perintah SQL.

Sistem menerapkan pola MVC dengan Model sebagai pengolah data, Controller sebagai pengatur proses aplikasi, dan View sebagai tampilan antarmuka. Operasi CRUD diterapkan pada resource Mahasiswa, Program Studi, dan Mata Kuliah.

Selain CRUD, sistem menggunakan JOIN untuk menampilkan nama Program Studi pada daftar mahasiswa. Konfirmasi JavaScript diterapkan sebelum proses penghapusan data untuk mencegah penghapusan secara tidak sengaja.

Sebagai Tugas Mandiri, ditambahkan fitur pencarian mahasiswa berdasarkan NIM atau nama menggunakan LIKE dan Prepared Statement. Pengujian menggunakan input ' OR '1'='1 menunjukkan bahwa input tersebut diperlakukan sebagai data pencarian, bukan sebagai perintah SQL.

Dengan demikian, implementasi praktikum menunjukkan penerapan PDO Prepared Statement, CRUD, MVC, JOIN, konfirmasi JavaScript, dan fitur pencarian pada Sistem Informasi Akademik.